



Saad Mughal | F2023-009

Total Marks: 100

Due 27 April 2026

DevOps and Cloud Computing

Assignment 2

**Sakila Flask Application | End-to-End
DevOps Pipeline**



Course Lecturer: Muhammad Ali

Section: A

Table of Contents

Section - 1.....	2
Section - 2.....	2
Section - 3.....	2
Section - 4.....	2
Section - 5.....	2
Section - 6.....	3

Part D: Reflection

Sakila Flask Application | End-to-End DevOps Pipeline

Section - 1

Git Workflow: Explain why branching matters in teams. Compare merge vs rebase: merge preserves history (safe for shared branches), rebase rewrites history (cleaner but dangerous on shared branches). Reference the conflict you resolved in A2.

Section - 2

Docker Optimization: List and explain all 6 fixes: slim base image (700MB → ~150MB), COPY requirements.txt before source (preserves cache), single RUN for pip (fewer layers), no hardcoded passwords (security), only expose 5000 (minimal attack surface), non-root USER (security).

Section - 3

Networking: Default bridge uses IPs only (no DNS). User-defined bridge (sakila-net) enables container-name DNS resolution via Docker's embedded DNS server. This is why MYSQL_HOST=sakila-db works in Compose.

Section - 4

Compose vs Manual: Compose solves: dependency ordering (depends_on), single-command lifecycle, shared network/volume definitions, env var management via .env. docker compose down stops containers but preserves volumes. docker compose down -v also removes volumes (all data lost).

Section - 5

CI/CD Design: Fail-fast ordering (lint→test→build) catches cheap errors early. Security scan doesn't block because vulnerabilities in base images are often out of our control, blocking would halt all deployments. Concurrency groups cancel stale runs, saving GitHub Actions minutes.

Section - 6

Production Readiness: Pick 3 from: secrets management (Vault or GitHub Secrets), monitoring (Prometheus + Grafana), Kubernetes for orchestration, Terraform for IaC, backup/DR strategy for the database.